

HYELLA CHRISTOPHER

COM314 ASSIGNMENT

BY: HYELLA CHRISTOPHER

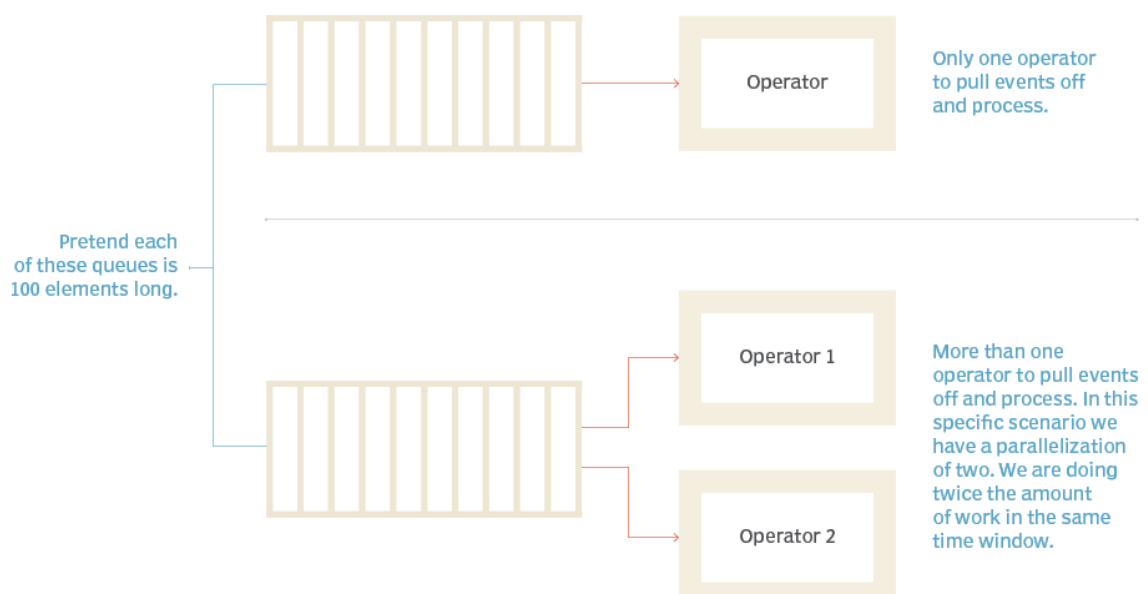
Q10a

What is pipelining?

Pipelining is the process of storing and prioritizing computer instructions that the processor executes. The pipeline is a "logical pipeline" that lets the processor perform an instruction in multiple steps. The processing happens in a continuous, orderly, somewhat overlapped manner.

In computing, pipelining is also known as *pipeline processing*. It is sometimes compared to a manufacturing assembly line in which different parts of a product are assembled simultaneously, even though some parts may have to be assembled before others. Even if there is some sequential dependency, many operations can proceed concurrently, which facilitates overall time savings.

Parallelization illustrated



Pipelining creates and organizes a pipeline of instructions the processor can execute in parallel. The pipeline is divided into logical stages connected to each other to form a pipelike structure. Instructions enter from one end and exit from the other. Pipelining is an ongoing, continuous process in which new instructions, or tasks, are added to the pipeline and completed tasks are removed at a specified time after processing completes. The processor executes all the tasks in the pipeline in parallel, giving them the appropriate time based on their complexity and priority. Any tasks or instructions that require processor time or power due to their size or complexity can be added to the pipeline to speed up processing.

Q10b

When we hear about **pipelining hazards** the first thing that comes to our mind is what are **pipeline hazards**? So, **pipeline hazards are simply any obstruction, condition or we can say any situation that is obstructing pipelines to work or act normally.**

Generally, there are mainly three **types of hazards**:

HYELLA CHRISTOPHER

1. Structural Hazard
2. Data Hazard
3. Control Hazard

We will discuss all the **hazards** one by one, let's start with structural hazard

1) Structural Hazard

When we try to do multiple or two different things using the same hardware in the same clock cycle this prevents the pipeline to work properly this is known as structural hazard.

To avoid this situation processor can use stalling in the pipelining.

Stall of one cycle will shift the pipeline to the one clock cycle until hazard can fully be avoided or eliminated.

This situation or hazard will not occur if we had separate data cache and instruction cache.

2) Data Hazard

In data hazard, read and write operations of shared variables by different instructions in a pipeline may lead to different kind of data dependencies such as,

- a. Read after write hazard
- b. Write after reading hazard
- c. Write after write hazard

They arise when an instruction depends on the result of previous instruction in a way i.e exposed by overlapping of instructions in the pipelining.

To avoid data hazard we can do

- a. Internal forwarding
- b. Stalling

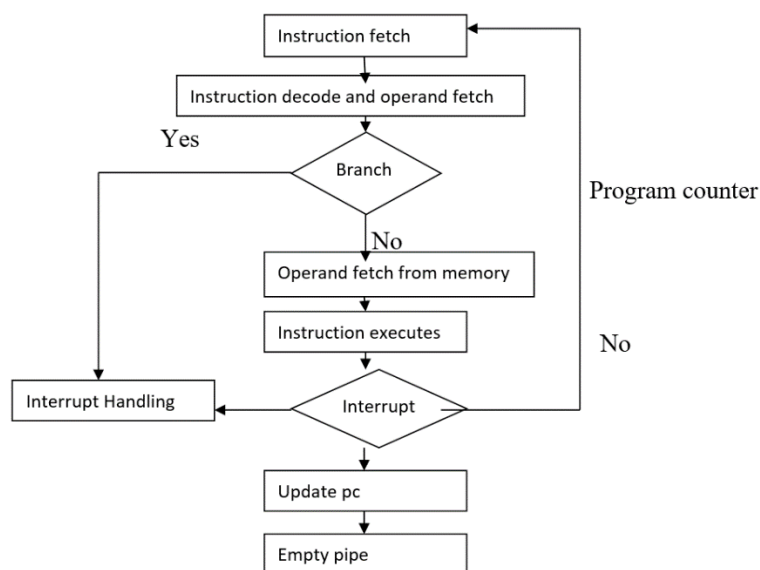
Internal forwarding- for avoiding the condition of reading after write hazard the latch which is placed right after ALU unit will be used as internal forwarding for transferring the output to the next instruction to the latch which is placed before the decoding unit.

Note: These are some other important terms related to data hazard:

- i. **Data dependency**
- ii. **Flow dependency** - Register R1 is loaded by I1 and then used by I2. Hence, the result of one register after executing instruction may affect the operand of that register.
- iii. **Output dependency** - when two instructions want to write the same time.

3) Control Hazard

In simple words, we can say that control hazard occurs when we need to find the main root of any branch instruction, and cannot proceed further with another instruction till we get to root of that instruction.



In this article, we have briefly discussed the **different pipelining hazards**. I hope that till now you

HYELLA CHRISTOPHER

have got a clear idea about the **pipelining hazards**. For any queries shoot your questions in the comment section.

HYELLA CHRISTOPHER

Q10c

Ingress/Egress Optimization:

Let's start with the optimizations you as a data engineer would need to perform for data to be read into or written by a pipeline.

Data formats, Compression and Data Type Conversions:

What are the formats of data and types of compression that the pipeline can support out of the box or with minimal code?

We often see data engineers writing manual or custom parsing routines to support common data formats, compression, and to do data type conversions. Doing this for every pipeline can be highly error-prone and time-consuming. Embracing automation for these parsing routines is critical to keep pipelines running and accurate.

Optimize for speed of ingress/egress:

Does the pipeline utilize information about the structure and storage of the data at the source and target to optimize for speed of ingress/egress?

For example, when deciding if a record needs to be processed or not, does it analyze metadata at the source object (such as a file, table or data partition) level or at the row level? If data is being read from a proprietary system such as an RDBMS or an MPP database, is the pipeline leveraging the technical advantages provided by these systems, such as parallelism, sharding, out-of-the-box change data capture, and native connectivity? Optimizing for speed of ingress/egress often has the most impact on the overall probability of pipelines meeting SLAs for data delivery to downstream systems.

Transformation Optimization:

For large initial loads or backfills:

While building a pipeline, there are usually a series of transformations performed on the data such as filtering, aggregating, sampling, joining and windowing. These transformations need to be optimized for the context of the data structures and the business logic. At this step, you need to perform several tasks:

- Design partitioning strategies
- Optimize transformation design
- Filtering data early on in the pipeline to reduce overall data movement
- Using the right data types for intensive operations
- Forward projection of only necessary columns
- Redistribution of data across partitions to ensure both performance and accuracy of the results

For Incremental loads:

In addition to the optimizations developed for the backfills, a separate set of optimizations need to be built for incremental updates to the dataset. A few examples are:

- Storage and usage of column statistics for incremental operations to reduce the amount of data scanned for repetitive transformations being run for every incremental update to the dataset.
- Partitioned joins to restrict the amount of data going into a join transformation to only process the updated data.

HYELLA CHRISTOPHER

Embracing repeatable and consistent processes to optimize transformations contributes the most to data engineering efficiency, as this can be time-intensive, and sadly most of it tends to be highly contextual and exclusive to each pipeline.

Resource Optimization:

For Independent Pipeline Deployments:

We're going to assume your choice has been made about on-prem, cloud, data processing platform, and other physical considerations for compute. When deploying a pipeline for the first time, based on the volume of data and the types of transformations, you could decide the number of compute instances, memory, storage, cache, data sharding, and other resource related artifacts. Usually this is done through a series of trials and errors before arriving at the optimal configuration that would allow the pipeline to complete in a reasonable amount of time and cost. This becomes an ongoing activity for pipelines that tend to be frequently updated both in terms of logic as well as addition of new data sets.

For Multi-tenant Deployments:

Once a strategy has been developed to optimize independent pipelines for resources, your organization's data platform will usually have several teams deploying these pipelines. If not well planned, the ensuing resource contention can cause disruption and delivery issues across these teams. In this case, a separate strategy for optimizing resources will need to be developed factoring in each pipeline's criticality to the business, data freshness needs for the business teams, service level requirements, and overall compute budgets assigned to individual teams or business units as part of an overall IT infrastructure strategy.

REFERENCES

Perez Garcia, Pablo (2018). Pipeline DSL a dsl to create a CI/CD pipeline for your projects. ISBN 978-0-134-69147-3.

Michael J. (2004). Parallel Programming in C with MPI and openMP. Dubuque, Iowa: McGraw-Hill Professional. ISBN 0072822562.

Pogonyi, Roland (February 2021). "What is a Data Pipeline?". Retrieved March 11, 2021.